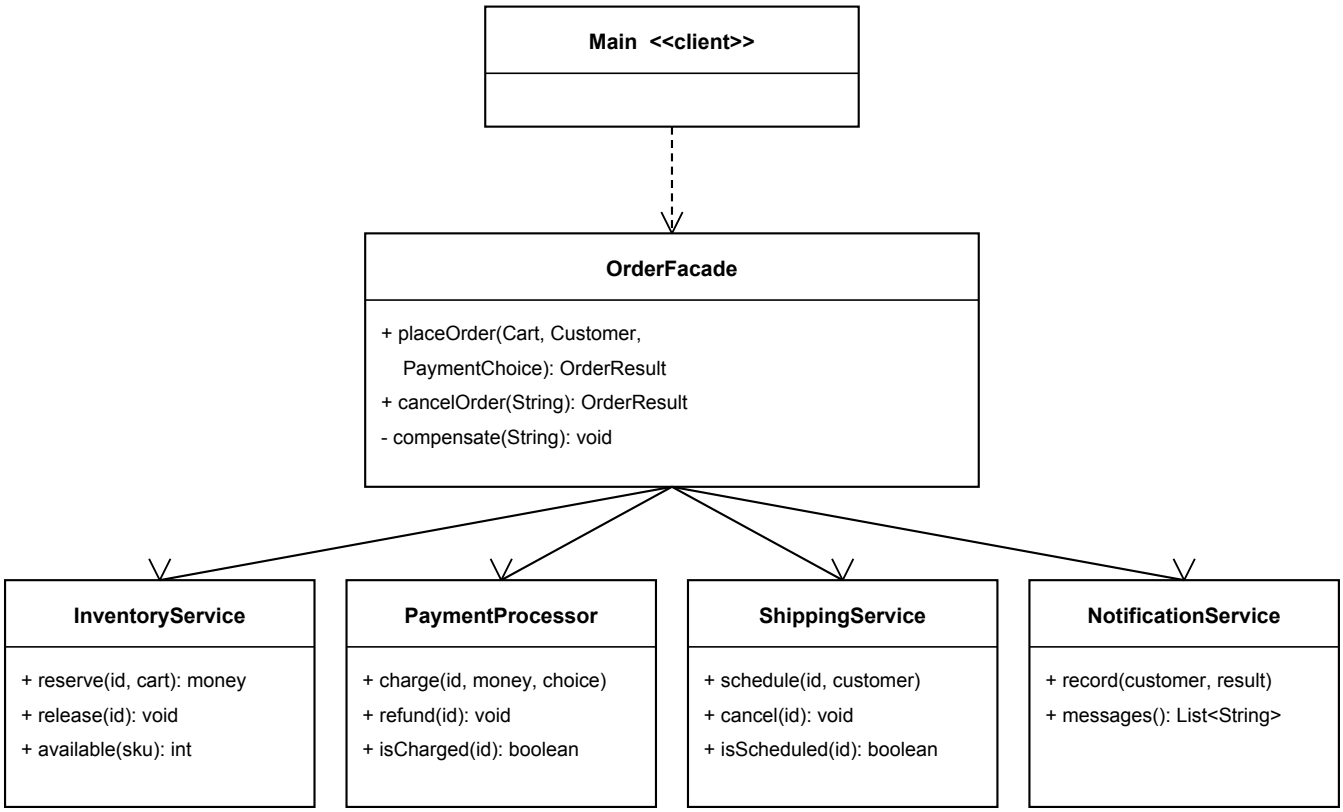
Assignment 4 Facade Pattern

Option B Online Order Processing | Prepared for Nurdaulet Mukhitden | SE-2537

AI-generated study reference. All payments, shipments and notifications are simulated in memory. No external transaction, email or order is created.

A small API for an ordered workflow

OrderFacade exposes placeOrder(cart, customer, choice) and cancelOrder(orderId). The client does not create or call subsystem services. InventoryService validates every cart line before reserving stock and computes the total; PaymentProcessor records a simulated charge; ShippingService schedules a simulated shipment; NotificationService records a confirmation.



Subsystems have no OrderFacade reference. Cart, Customer and OrderResult are immutable API data types.

Failure behavior

Placing an order reserves stock, charges the total, schedules shipping and records a confirmation in that order. A payment decline or shipping failure triggers compensation: cancel shipping, refund payment, then release stock. Stock release and refund are idempotent. Repeated cancellation returns the existing cancelled result without restoring stock twice.

Five Clean Code examples

A meaningful simplified operation

src/OrderFacade.java:31

```
    public OrderResult placeOrder(Cart cart, Customer customer, PaymentChoice choice) {
```

The caller supplies domain data and gets an immutable OrderResult. It does not need to know the reservation, charging, shipment and notification sequence.

Single responsibility in the facade

src/OrderFacade.java:37

```
            BigDecimal total = inventory.reserve(id, cart);
            payments.charge(id, total, choice == PaymentChoice.ACCEPTED
                    ? PaymentProcessor.Decision.APPROVE : PaymentProcessor.Decision.DECLINE);
            String tracking = shipping.schedule(id, customer);
```

The facade coordinates services. Inventory calculates the amount; payment and shipping maintain their own state. Those responsibilities are not reimplemented in the facade.

Keeping subsystem details behind the API

src/OrderResult.java:3

```
public record OrderResult(String orderId, BigDecimal total,
                          String trackingNumber, String status) { }
```

The returned value contains an order ID, total, tracking reference and status. Mutable service objects and internal maps are not returned to the ordinary client.

Focused state-changing operations

src/InventoryService.java:34

```
    public void release(String orderId) {
        Cart cart = reservations.remove(orderId);
        if (cart != null) {
            cart.quantities().forEach((sku, quantity) -> stock.put(sku, stock.get(sku) + quantity));
        }
```

Releasing a reservation is handled in one small service operation. Removing the reservation first makes repeated release harmless.

Error handling and compensation

src/OrderFacade.java:45

```
        } catch (OrderException failure) {
            compensate(id);
            throw new OrderException("Cannot place order: " + failure.getMessage(), failure);
        }
```

A coordination failure triggers compensation before the caller receives a domain exception. The original exception remains available as the cause for explanation and debugging.

Evidence and design limits

Scenario	Result
Successful order	Total 3300.00; stock and charge match
Cancellation	Stock restored; charge and shipment removed
Repeated cancellation	No duplicate release or notification
Payment decline	Reservation released; no charge or shipment
Shipping failure	Payment refunded and reservation released
Insufficient stock	No cart line is partly reserved
Invalid input	Unknown order, empty cart, invalid customer and negative quantity rejected

FacadeChecks was compiled and run on Amazon Corretto 17.0.13 with --release 17. All ten checks passed. demo-output.txt contains actual check output. Main is a separate client demonstration using only the facade and immutable domain values.

What the facade hides

The facade hides service construction for ordinary clients, call ordering, order-ID allocation, internal transaction records and compensation. An advanced client can still directly use the public InventoryService, PaymentProcessor, ShippingService and NotificationService APIs. None of these classes stores an OrderFacade reference.

Limits of this example

This is a single-threaded in-memory teaching application. Data disappears when the process stops. Compensation is reliable within these deterministic local services; it is not a distributed transaction protocol. Real systems need persistence, concurrency control and recovery when a compensation step itself fails. The demo does not prove those properties.

Submission and defense

LMS deadline: 18 October 2026 at 23:59. Defense: the last practice class of Week 7. GitHub publication and its URL are still required for an actual submission. The course limits AI to reference examples and clarification; this generated reference must not be presented as independently authored student work. Nothing has been uploaded to Moodle.

References

Course Assignment_4_Facade_Pattern_Instructions.docx, sections 1-9. Refactoring.Guru, Facade: https://refactoring.guru/design-patterns/facade (accessed 17 October 2026).